

Módulo 2. Conceptos básicos de programación

Introducción

Video de inmersión

Unidad 1. Representación gráfica de los procesos

Representar algoritmos o procesos puede ser algo difícil. No siempre se pueden encontrar las palabras adecuadas y, en muchos casos, sería muy difícil de entender o muy largo de leer si solamente se utilizan palabras. Es por ello que se creó un lenguaje de símbolos, para poder representar gráficamente los procesos mediante diagramas.

Otra forma de comunicar los procesos o algoritmos es mediante un pseudocódigo, que puede definirse como un conjunto de sentencias que no corresponden a ningún lenguaje de programación específico, pero que denotan la lógica que se debe seguir por dicho algoritmo.

Tema 1. Origen y características

Inicialmente, el proceso de programación se realizaba en lenguajes de programación más cercanos a la computadora, lenguajes de más bajo nivel (lenguaje máquina). Los datos están codificados en sistema binario, y no es necesaria la traducción. Entre las características del lenguaje máquina, cabe destacar las siguientes:

Es dependiente de los recursos de la computadora; por lo tanto, para programar, el programador debe conocer la arquitectura sobre la que se programa.

El programador se encarga de verificar que no existan errores sintácticos, pues no existe un compilador que los detecte.

El programador trabaja directamente con direcciones de memoria

El lenguaje ensamblador surgió como evolución natural, donde a cada secuencia de ceros y unos se le asocia un nombre nemotécnico. Estos nombres necesitan traducción, que se realiza mediante un programa que se llama como el lenguaje: ensamblador. Aunque fue un gran avance, todavía es necesario conocer cómo está constituida y qué recursos tiene la computadora.

Más tarde, se fueron asociando nombres a conjuntos de instrucciones que realizaban una tarea compleja determinada y programaban de manera independiente la computadora donde se iba a ejecutar el código. Nacen los lenguajes de alto nivel y se encuentran más cercanos a la forma de pensar de los humanos que al lenguaje que entiende la máquina.

La programación, de la mano de los lenguajes, fue evolucionando gracias a cuatro causas o motores que la impulsan. Estas son las siguientes:

Abstracción: es el proceso mental por el que el ser humano extrae las características esenciales de algo e ignora los detalles superfluos. En programación, el término se refiere al énfasis en el qué, más que en el cómo.

Es esencial para modelar el mundo real. Al principio, se hacían programas pensando como una computadora sobre la cual corrían. En la actualidad, se solucionan los problemas sin conocer la máquina donde va a ser ejecutado el programa.

Encapsulación: es el principio sobre el cual enmascaramos funcionalidad en programación, es esencial para reutilizar un código. Se ocultan los detalles de cómo está hecho un programa, pero se conoce el modo de funcionamiento, cuáles son sus entradas y qué salida produce, de forma tal que se pueda reutilizar en cualquier otro programa.

Modularidad: es el proceso de descomposición de un sistema en un conjunto de elementos poco acoplados (independientes) y cohesivos (con significado propio). Es esencial para abordar la resolución de problemas extensos o complicados de un modo más simple y organizado.

Jerarquía: es el proceso de estructuración por el que se organizan un conjunto de elementos en distintos niveles, atendiendo a determinados criterios (responsabilidad, composición, etc.). Es decir, la jerarquización implica organizar programas de acuerdo con rangos, clases o categorías.

A medida que se fueron añadiendo nuevas características al proceso y a las herramientas, fueron surgiendo múltiples estilos de programación.

Características de la programación

Es importante tener algunas consideraciones para que la programación sea de buena calidad y los resultados sean o se acerquen lo más posible a los esperados, en tiempo y forma. Para esto, se pueden enunciar las siguientes características que se deben tener en cuenta durante la programación de aplicaciones:

≡ Eficacia

≡ Eficiencia

≡ Integridad o completitud

≡ Claridad

≡ Portabilidad

Tema 2. Conceptos fundamentales

La generación de un programa o un software en particular necesita de un conjunto de reglas, actividades y procedimientos para lograr realizar los algoritmos y cumplir su objetivo. Esta metodología llamada “ciclo de desarrollo del software” consta de una serie de pasos lógicos secuenciales denominados “fases”. Si bien esas fases pueden tomar diferentes nombres según los autores que las describen, conceptualmente se refieren a las mismas actividades y son las siguientes:

1. Definición de necesidades / Identificación del problema / Investigación preliminar
2. Análisis/ Interpretación del problema / Determinación de requerimientos
3. Diseño de la solución / Diseño del sistema
4. Codificación / Desarrollo del Sistema
5. Prueba, depuración y documentación
6. Implementación / Validación / Implantación

Es importante conocer cada una de estas fases, ya que están íntegramente relacionadas. El resultado que producen sirve de entrada para el comienzo de la siguiente etapa. Las fases ligadas al programador dependen de la metodología de desarrollo que se use. Cada rol involucrado en este proceso debe conocer e interactuar en cada una de las etapas.

Definición de las necesidades o Identificación del problema

Conocer el problema e identificar las necesidades es la primera consideración. Para ello debemos saber quién es el usuario final, cómo opera con el sistema, qué resultados se esperan, etc. Una vez identificadas las necesidades, se deberá evaluar la factibilidad de cumplimentarlas.

Análisis e interpretación del problema

Es la comprensión completa del problema con sus detalles. En este punto empezamos a definir el alcance de la solución, es decir, qué se va a desarrollar y qué no. Nos involucramos con el problema y buscamos todas las fuentes de información necesarias para solucionarlo. Como resultado de este análisis exhaustivo debemos realizar la determinación de las entradas y salidas del proceso de software, cómo funcionará el programa, qué datos se necesitan y qué salida se deberá generar. Los requisitos para lograr una solución eficaz y eficiente.

Diseño de la solución

Es momento de comenzar a diseñar y modelar los algoritmos. Una computadora no tiene la capacidad para solucionar más de lo que se le indica en los algoritmos. Estos algoritmos indican las instrucciones para que la máquina ejecute. La información proporcionada al algoritmo constituye su entrada, y la información producida por el algoritmo constituye su salida.

Los problemas complejos se pueden resolver más eficazmente cuando se dividen en subproblemas más fáciles de solucionar que el original. A la descomposición del problema original en subproblemas más simples le sigue la división de estos subproblemas en otros más simples aún. Estas divisiones se realizan hasta que los subproblemas resultantes sean lo más pequeños posibles y permitan realizar sus respectivos diseños.

Existen muchas formas de plasmar la solución analizada. Entre las más comunes se encuentran dibujar un diagrama de flujo y escribir un pseudocódigo. En algunos casos, para entendimiento de la persona que solicita la resolución del problema, es necesario emplear ambas metodologías.

Codificación

Este paso consiste en empezar a escribir el código del programa, es decir, expresar la solución en un lenguaje de programación, traducir la lógica del resultado de la fase anterior a un lenguaje propiamente dicho. Como ya hemos señalado, un lenguaje de programación es un conjunto de reglas que proporciona una forma de instruir a la computadora qué operaciones realizar.

Prueba y depuración

Algunos expertos insisten en que un programa bien diseñado se puede escribir correctamente la primera vez. De hecho, afirman que hay formas matemáticas de demostrar que un programa es correcto. Sin embargo, las imperfecciones del mundo existen, por lo que la mayoría de los programadores se acostumbran a la idea de que sus programas recién escritos probablemente tengan algunos errores. Esto es un poco desalentador al principio, ya que los programadores tienden a ser personas precisas, cuidadosas y orientadas a los detalles, personas que se enorgullecen de su trabajo. Aun así, hay muchas oportunidades para introducir errores en los programas. Por esta razón, los programas deben probarse en un ambiente controlado, dedicado a tal fin.

La depuración es un término usado ampliamente en programación, significa detectar, localizar y corregir errores, generalmente, ejecutando el programa. En muchas ocasiones esta fase se realiza de manera reiterada, hasta que no se encuentren errores en el código programado. La prueba o verificación se puede realizar de forma manual o automática, incluso existen diversas herramientas para realizar esta automatización. Para probar los programas utilizamos datos y flujos de prueba diseñados previamente; en la jerga informática se los conoce como casos de prueba. Estos casos de prueba se diseñan conociendo el problema que se tiene que solucionar, se planifican los datos y los flujos cuidadosamente para asegurarse de verificar cada parte del programa.

Documentación

En realidad, esta actividad no puede ser catalogada como una etapa del proceso de desarrollo de software, por lo general, se puede presentar en la etapa de codificación o en la etapa de verificación. Documentar es un proceso continuo y necesario. Sin embargo, como les pasa a muchos programadores, podés sentirte ansioso por realizar actividades más emocionantes centradas en la programación. La documentación es una descripción detallada, por escrito, del ciclo de programación y hechos específicos sobre el programa. Los materiales típicos de documentación del programa incluyen el origen y la naturaleza del problema, una breve descripción narrativa, herramientas lógicas como diagramas de flujo y pseudocódigos, descripciones de registros de datos, listas de programas y resultados de pruebas.

Los comentarios en el programa en sí también se consideran una parte esencial de la documentación. Muchos programadores documentan mientras codifican. En algunos casos esta práctica no es recomendada en exceso porque puede aumentar el tamaño del programa y, por lo tanto, no ser efectivo para la funcionalidad.

El programador inteligente continúa documentando el programa a lo largo de su diseño, desarrollo y prueba. Se necesita documentación para complementar la memoria humana y para ayudar a organizar la planificación del programa. Además, la documentación es importante para comunicarse con otras personas interesadas en el programa, especialmente, otros programadores que pueden ser parte de un equipo de programación.

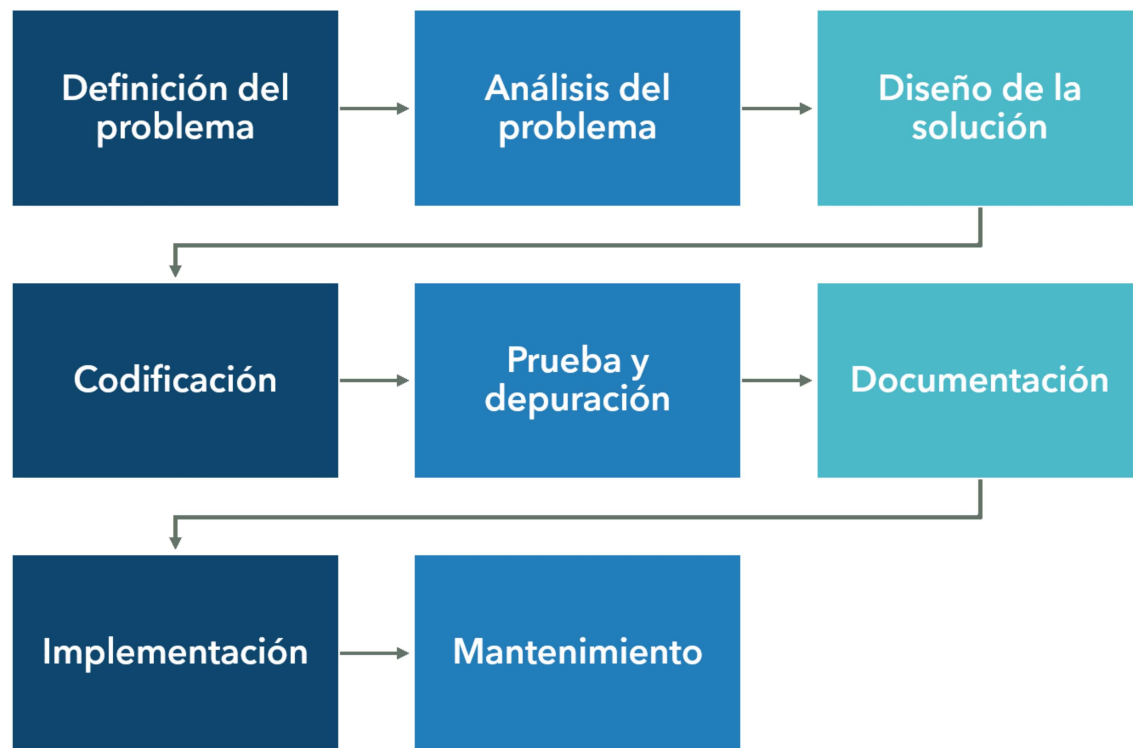
Implementación

El programa ya probado, revisado y mejorado se considera terminado y puede utilizarse con un alto grado de confianza para resolver los problemas que dieron origen a su creación. Si se está automatizando alguna tarea manual, esta última se desecha para emplear solo el programa. Los programas codificados se implantan en un ambiente productivo para su uso masivo, en muchos casos esta etapa se conoce como la liberación del producto.

Mantenimiento

Es la fase de mayor duración, permite adaptar el desarrollo a cambio de necesidades del negocio o tecnológicas. Cuando se detectan estas alteraciones en los requerimientos estipulados con anterioridad, se debe realizar el mantenimiento del software. Siempre es preferible realizar un mantenimiento preventivo, por lo cual se debe estar atento a los cambios de necesidades.

Figura 2: Fases de un proyecto de software



Fuente: elaboración propia.

A continuación, desarrollamos algunas de las características que deben cumplir los programas con respecto a su programación:

≡ Usabilidad

≡ Mantenibilidad

≡ Fiabilidad

Tema 3. Diagrama de flujos y pseudocódigo

El pseudocódigo es considerado un lenguaje de alto nivel, es mucho más entendible para los humanos que un lenguaje de programación. Sin embargo, cuenta con una estructura similar a un código.

Los pseudocódigos no se compilan ni interpretan por ninguna computadora. Su propósito es, simplemente, representar un algoritmo o un proceso mediante una sintaxis similar a la presente en los lenguajes de programación. Sin embargo, se han creado aplicaciones que permiten simular la ejecución de nuestro algoritmo. Nosotros utilizaremos la aplicación PSeInt para poder ejemplificar

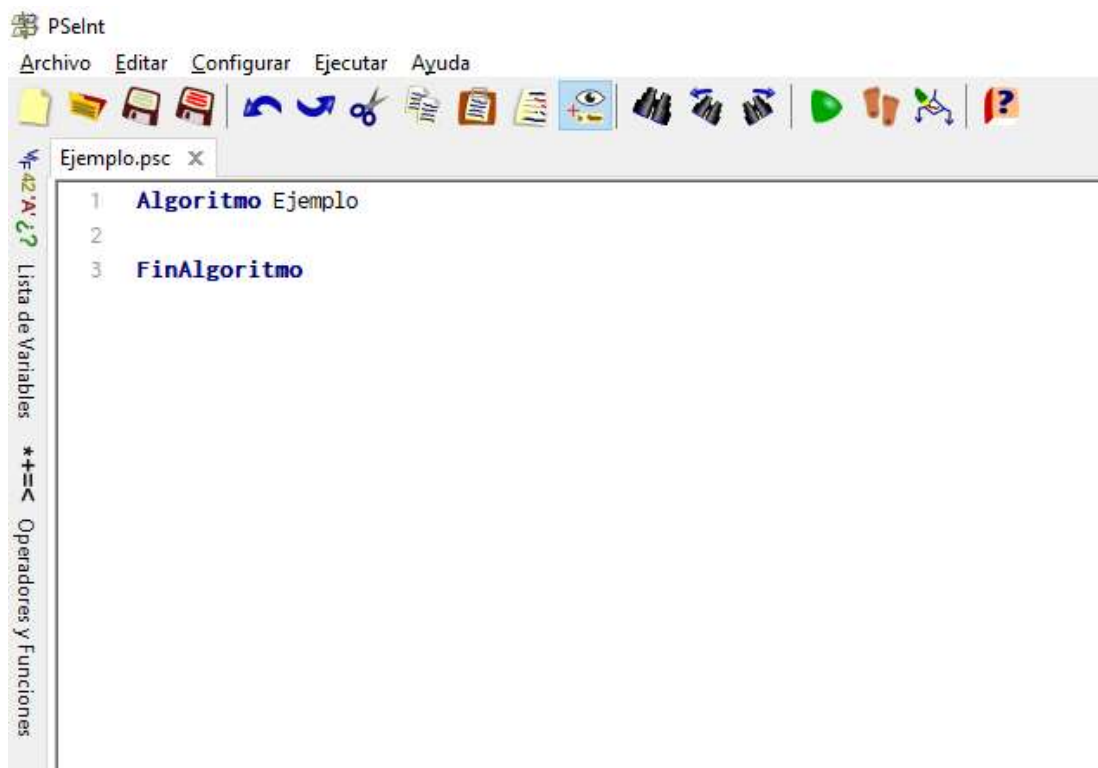
cada una de las secciones del algoritmo en pseudocódigo. Podés encontrarla en el siguiente link:
<http://PseInt.sourceforge.net/>

Inicio / Fin

Como revisamos en el módulo anterior, un algoritmo es finito, es por eso que siempre debe tener un comienzo y un fin.

En PSeInt, un algoritmo comienza con su nombre propiamente dicho, esto se ve reflejado en el inicio y por un FinAlgoritmo al terminar el código.

Figura 3: Estructura de un algoritmo PSeInt



Fuente: elaboración propia con base en el software PSeInt (Novara, 2021).

Variables y tipos de datos

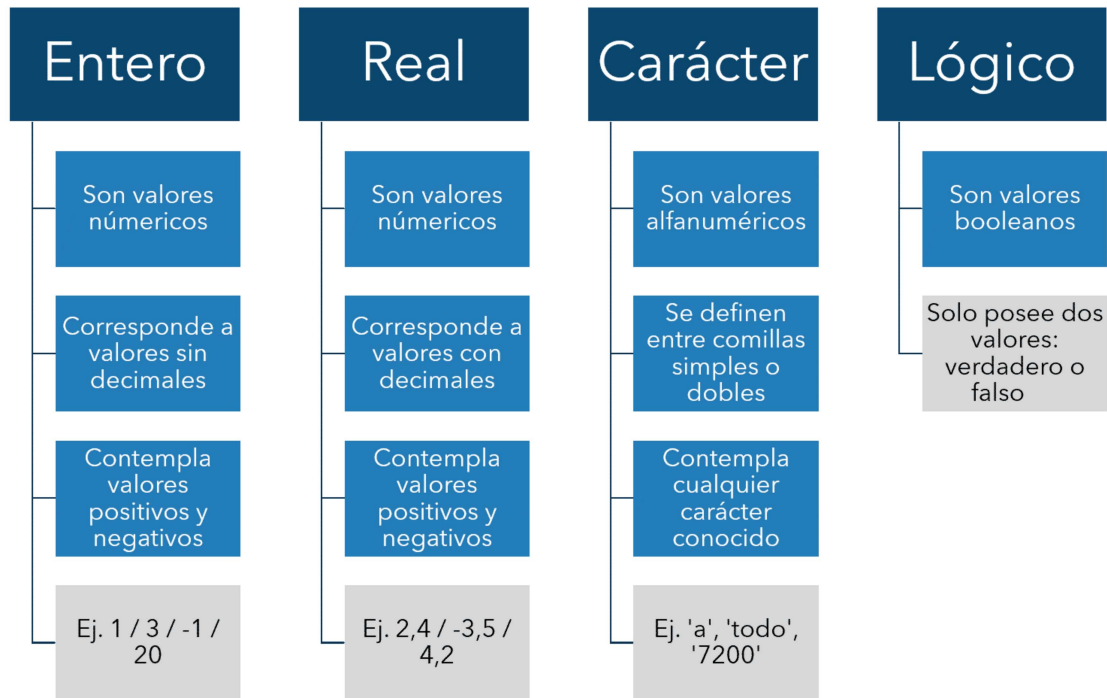
Es necesario declarar las variables que utilizaremos. Una variable será un medio para almacenar valores en forma temporal, los que podrán cambiar a lo largo del algoritmo. Al finalizar dicho programa las variables se borran de memoria y finalizan su existencia.

Por ejemplo, si queremos manipular o realizar operaciones con valores entregados por un usuario, estos deben quedar almacenados en variables. Estas se definen con un nombre, que nos permitirá identificarlas a lo largo del algoritmo.

Al declarar la variable, debemos indicar el tipo de dato que almacenará. Aunque existen algunos lenguajes que no lo necesitan, la mayoría sí lo requiere y es bueno cuando se comienza a aprender.

Los tipos de datos básicos que puede tomar una variable son los siguientes:

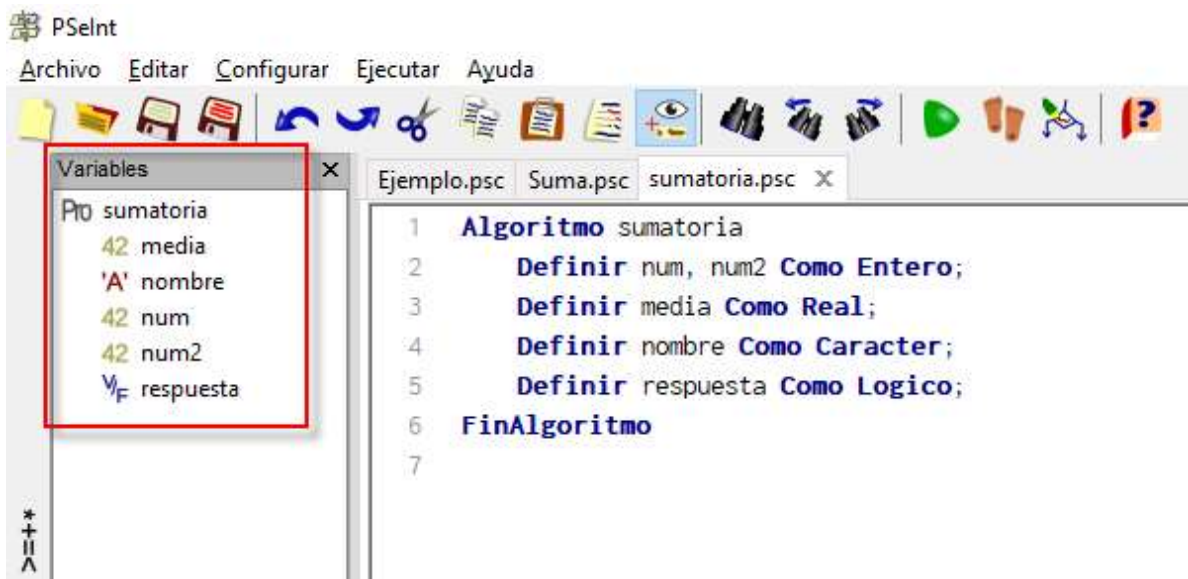
Figura 4: Tipos de datos



Fuente: elaboración propia.

Dentro del PSeInt, las variables y sus tipos de datos se declaran al comienzo del algoritmo. Más adelante, veremos que en los lenguajes de programación las variables se declaran de acuerdo con su alcance en el código.

Figura 5: Declaración PSeInt



Fuente: elaboración propia con base en el software PSeInt (Novara, 2021)

Comentarios

Los comentarios son textos que nos permiten incluir notas en nuestro algoritmo o código. Estos son ignorados al ejecutar las instrucciones. En este caso, los utilizaremos para indicar las secciones.

En PSeInt, los comentarios en el código se expresan anteponiendo los caracteres `//` al texto que se va a comentar. En el ejemplo anterior, el comentario nos entrega una referencia de la sección del código

`//Definir Variables`

Leer

La instrucción Leer, en PSeInt, permite recuperar datos que ingresa el usuario.

Figura 6: Lectura PSeInt

```

1  Algoritmo sumatoria
2      // Definir variables
3      Definir num, num2 Como Entero;
4      Definir media Como Real;
5      Definir nombre Como Caracter;
6      Definir respuesta Como Logico;
7
8      //Lectura de variables
9      Leer num;
10     Leer num2;
11 FinAlgoritmo
12

```

Fuente: elaboración propia con base en el software PSeInt (Novara, 2021).

Escribir

La instrucción Escribir en PSeInt permite mostrar datos al usuario.

Figura 7: Escritura PSeInt

```

1  Algoritmo sumatoria
2      // Definir variables
3      Definir num, num2 Como Entero;
4      Definir media Como Real;
5      Definir nombre Como Caracter;
6      Definir respuesta Como Logico;
7
8      //Lectura de variables
9      Leer num;
10     Leer num2;
11
12     //Generar salida
13     Escribir num + num2
14 FinAlgoritmo
15

```

Fuente: elaboración propia con base en el software PSeInt (Novara, 2021).

Se necesita un algoritmo que obtenga la información de una persona. Identificá el tipo de datos de la siguiente lista de atributos y declaralo usando pseudocódigo. Luego imprimí el nombre en conjunto con el apellido.

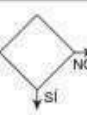
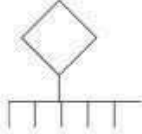
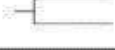
- Nombre
- Apellido
- Edad
- EsCasado (verdadero o falso)
- EsEstudiante (verdadero o falso)
- Dirección
- Solución
- Definir nombre como Carácter.
- Definir apellido como Carácter.
- Definir edad como Entero.
- Definir esCasado como Lógico.
- Definir esEstudiante como Lógico.
- Definir dirección como Carácter.
- Definir promedio como Real
- Diagrama de flujo

Un diagrama de flujo expresa, de manera gráfica, los pasos que se deben seguir y las decisiones que se van a tomar en un algoritmo o proceso específico.

Existen estándares que definen los símbolos que se deben utilizar en los diagramas de flujo. Dichos diagramas no siempre son algoritmos, pueden ser procesos de negocio, procesamiento electrónico de datos, entre otros.

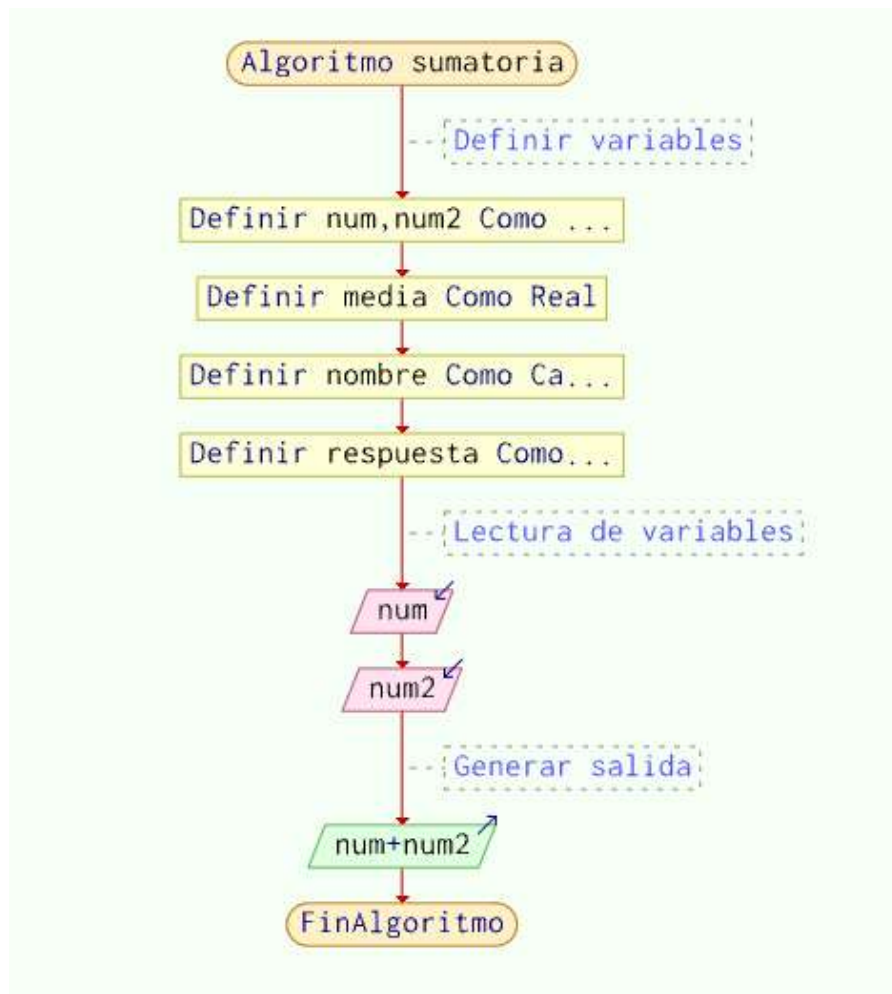
En este caso, utilizaremos los estándares ANSI (American National Standards Institute) para la diagramación administrativa, que son los utilizados para la diagramación de algoritmos.

Tabla 1: Simbología ANSI

Símbolos principales	Función
	Terminal (representa el comienzo, “inicio”, y el final, “fin” de un programa. Puede representar también una parada o interrupción programada que sea necesario realizar en un programa).
	Entrada/Salida (cualquier tipo de introducción de datos en la memoria desde los periféricos, “entrada”, o registro de la información procesada en un periférico, “salida”).
	Proceso (cualquier tipo de operación que pueda originar cambio de valor, formato o posición de la información almacenada en memoria, operaciones aritméticas, de transferencia, etc.).
	Decisión (indica operaciones lógicas o de comparación entre datos —normalmente dos— y en función del resultado de la misma determina cuál de los distintos caminos alternativos del programa se debe seguir; normalmente tiene dos salidas —respuestas SÍ o NO— pero puede tener tres o más, según los casos).
	Decisión múltiple (en función del resultado de la comparación se seguirá uno de los diferentes caminos de acuerdo con dicho resultado).
	Conector (sirve para enlazar dos partes cualesquiera de un organograma a través de un conector en la salida y otro conector en la entrada. Se refiere a la conexión en la misma página del diagrama).
	Indicador de dirección o línea de flujo (indica el sentido de ejecución de las operaciones).
	Línea conectora (sirve de unión entre dos símbolos).
	Conector (conexión entre dos puntos del organograma situado en páginas diferentes).
	Llamada a subrutina o a un proceso predeterminado (una subrutina es un módulo independientemente del programa principal, que recibe una entrada procedente de dicho programa, realiza una tarea determinada y regresa, al terminar, al programa principal).
	Pantalla (se utiliza en ocasiones en lugar del símbolo de E/S).
	Impresora (se utiliza en ocasiones en lugar del símbolo de E/S).
	Teclado (se utiliza en ocasiones en lugar del símbolo de E/S).
	Comentarios (se utiliza para añadir comentarios clasificadores a otros símbolos del diagrama de flujo. Se pueden dibujar a cualquier lado del símbolo).

Fuente: Joyanes Aguilar, 2007, p. 72.

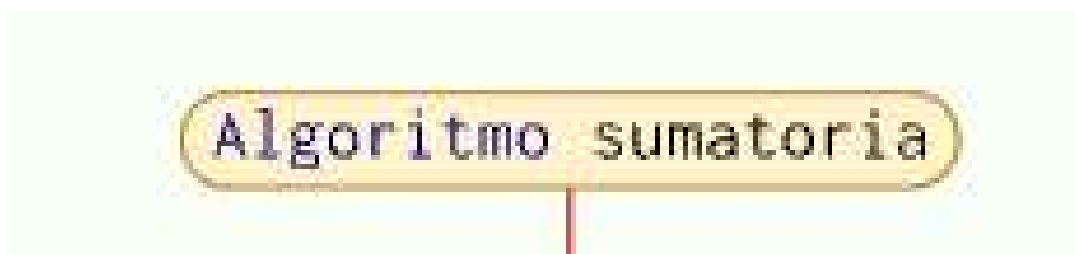
Figura 8: Diagrama de flujo PSeInt



Fuente: elaboración propia con base en el software PSeInt (Novara, 2021).

En el símbolo de Inicio, PSeInt indica el nombre del algoritmo.

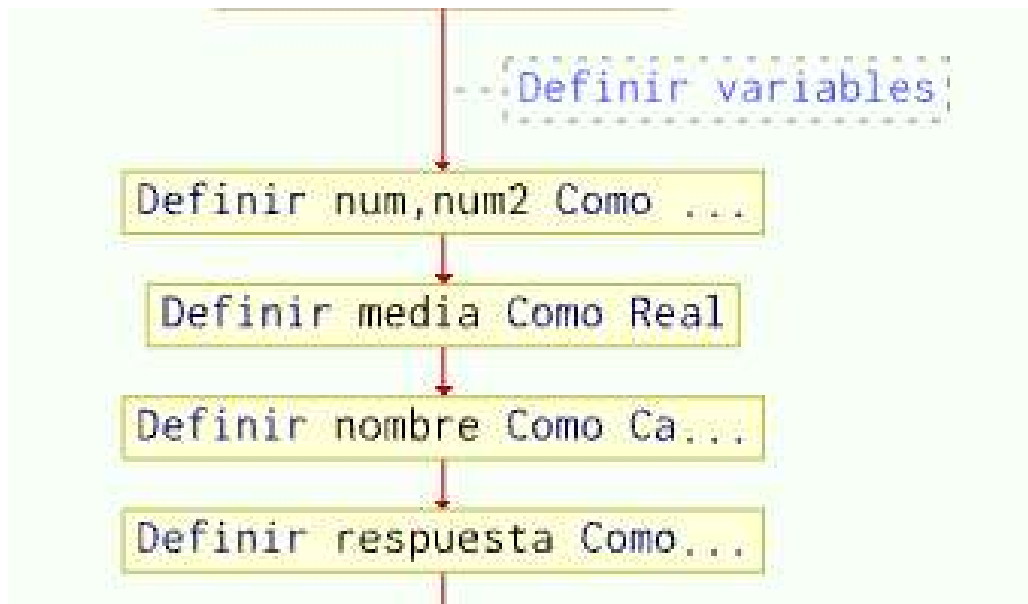
Figura 9: Inicio



Fuente: elaboración propia con base en el software PSeInt (Novara, 2021).

Las definiciones de las variables son identificadas como parte del proceso. En la imagen, se incluye el comentario que indica la sección del algoritmo.

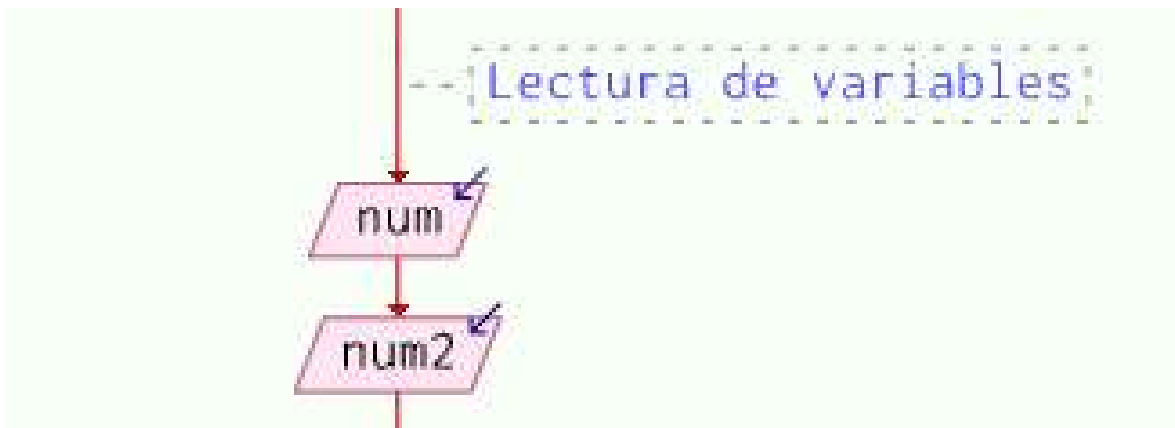
Figura 10: Proceso



Fuente: elaboración propia con base en el software PSeInt (Novara, 2021).

Las entradas de datos se representan, en PSeInt, con una flecha hacia el interior del símbolo, para indicar que son entradas, ya que los símbolos de entrada y salida son iguales. En la imagen, se incluye el comentario que indica la sección del algoritmo.

Figura 11: Entrada



Fuente: elaboración propia con base en el software PSeInt (Novara, 2021).

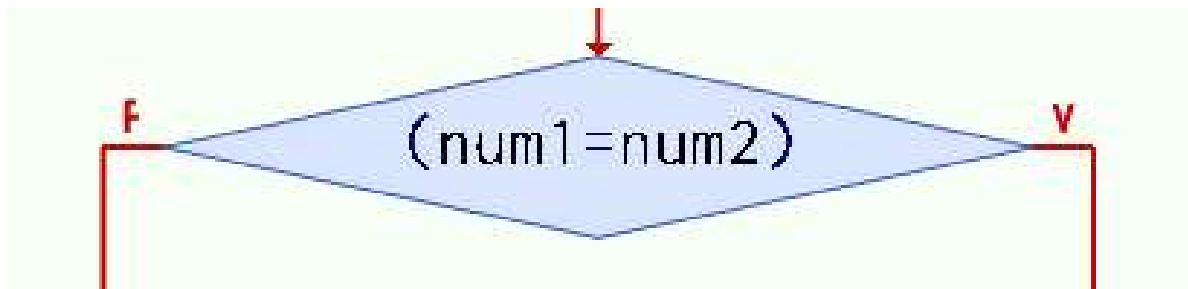
Las decisiones se representan a través de un rombo, donde se definen ambos caminos, verdadero y falso. Para los casos de decisiones iterativas, la diferencia es que, al terminar la serie de instrucciones, se retorna a la decisión en lugar de continuar secuencial.

Ejercicio

Continuá con el ejercicio de la suma ejemplificado en la figura 11, modificá la salida generada para

que, si la variable num es menor o igual a num2 imprima la suma, y en caso contrario imprima la resta.

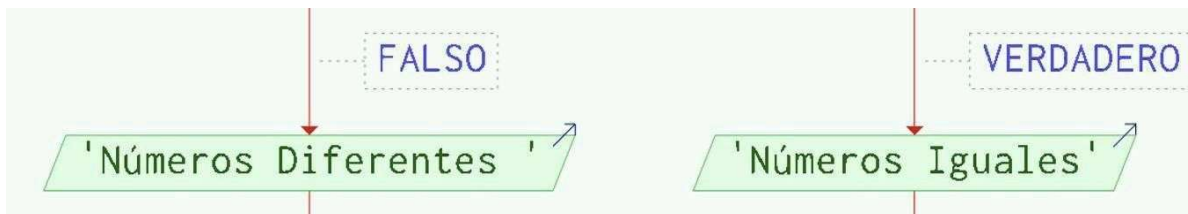
Figura 12: Decisión



Fuente: elaboración propia con base en el software PSeInt (Novara, 2021).

Las salidas de datos se representan, en PSeInt, con una flecha hacia el exterior del símbolo, para indicar que son salidas. En la imagen, se incluye el comentario que indica la sección del algoritmo.

Figura 13: Salidas



Fuente: elaboración propia con base en el software PSeInt (Novara, 2021).

El símbolo utilizado para el término del algoritmo es el mismo que se utiliza para el inicio, sin embargo, este indica específicamente que es el fin y desde él no salen nuevos caminos.

Figura 14: Fin



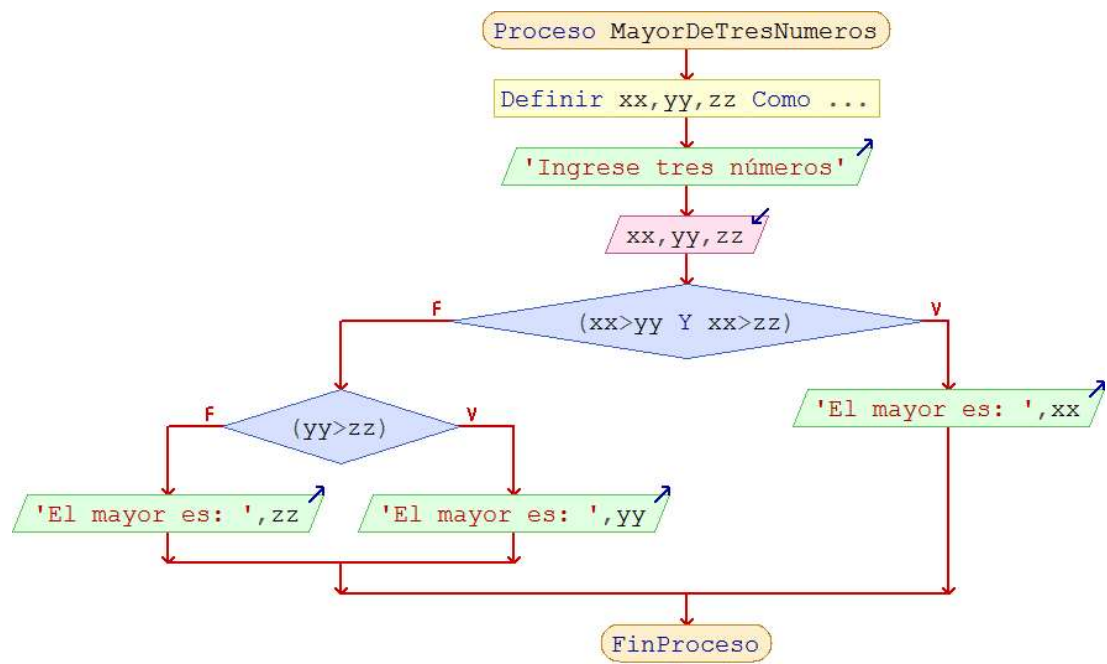
Fuente: elaboración propia con base en el software PSeInt (Novara, 2021).

Los diagramas de flujo pueden ser muy simples —como los vistos ahora— o muy complejos y largos. Todo depende del problema que se esté intentando resolver.

Ejercicios

En el siguiente algoritmo, indicá a qué elemento corresponde cada uno de los símbolos.

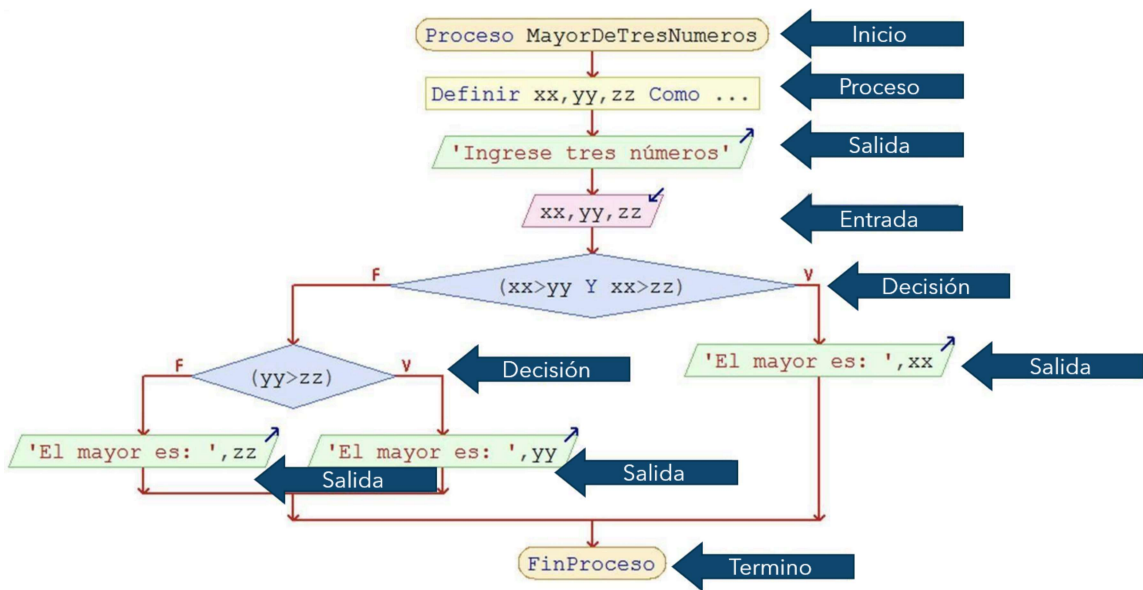
Figura 15: Ejercicio



Fuente: elaboración propia con base en el software PSeInt (Novara, 2021).

Resultado

Figura 16: Resolución



Fuente: elaboración propia con base en el software PSeInt (Novara, 2021).

Tema 4: Uso de variables y tipos de datos

Las variables son un elemento fundamental de la programación, dado que, más allá de lo que haga el sistema que estamos construyendo, siempre vamos a necesitar almacenar información en la memoria del computador. Por ese motivo a esas porciones de memoria que contienen los datos se les asignan nombres simbólicos (identificadores), y así conforman lo que un programador maneja como variables dentro de su código.

Casi todos los datos que se manejan en nuestro programa se almacenan en variables.

Se debe concebir “variable” como un contenedor de información.

Tomemos como ejemplo una definición de una variable de tipo cadena de caracteres (string) en C++:

Figura 17: Variable de tipo texto en C++

```
1
2  #include <iostream>
3
4  using namespace std;
5
6  int main()
7  {
8      char saludo[100] = "Hola Mundo";
9
10     cout<<saludo;
11
12     return 0;
13 }
```

Fuente: elaboración propia.

El tipo de datos string corresponden a C++ no es un tipo de dato nativo, por lo cual, para utilizarlo necesitamos incluir en nuestro programa la librería string. A continuación, un ejemplo de su utilización:

Figura 18: Incluir string en el programa

```
#include <stdio.h>
#include <iostream>
#include <string>
using namespace std;

int main()
{
    string saludo = "Hola Mundo";

    cout << saludo;

    return 0;
}
```

Fuente: elaboración propia.

Las variables siempre tienen un tipo de dato asociado, que establece cómo el lenguaje tiene que tratar el dato contenido dentro de esa variable; por ejemplo, no es lo mismo manipular un número que una palabra.

Tipos de datos

Existen diferentes tipos de datos, los principales son los siguientes:

Integers: este tipo de datos se utilizan cuando queremos almacenar un número sin decimales (un número entero). Por ejemplo, es lo que usamos si queremos calcular la suma de $100 + 300$.

Float: el tipo float permite la manipulación de números con decimales. El número 12,25 sería de tipo float.

Double: las variables de este tipo, al igual que las del tipo float, permiten manipular números con decimales. La principal diferencia es la precisión. Como el nombre lo indica, el tipo de dato double tiene una precisión doble que el tipo de dato float (2x). En general, un double tiene 15 dígitos decimales de precisión, mientras que el tipo de dato float tiene solo 7.

Character: representa un único carácter, que puede ser un número, una letra o un símbolo.

String: representa cadenas de caracteres. Es utilizado cada vez que necesitamos manipular o

almacenar cadenas con letras, números y símbolos. Un texto, por ejemplo, se debe almacenar con este tipo de dato. En C++ se usa por medio de la librería string.

Boolean: puede tomar solamente los valores “true” (verdadero) o “false” (falso).

Figura 19: Tipos de datos en C++

```
string varstring = "Hola Mundo";  
int varint = 1;  
float varfloat = 10.25f;  
double vardouble = 5.5;  
bool varbool = true;  
char varchar = '!';
```

Fuente: elaboración propia.

Según el tipo de dato, las variables serán manejadas y procesadas dentro del programa de diferentes modos. Con variables numéricas, se podrán realizar, por ejemplo, conteos y operaciones matemáticas; sobre strings o cadenas de caracteres, se podrán realizar concatenaciones o divisiones en cadenas más pequeñas, etcétera.

Constantes

Algunos datos necesarios tendrán información almacenada que no cambiará a lo largo del programa. En estos casos, es conveniente declarar ese dato como una constante en lugar de una variable.

Una constante es un valor que no puede ser alterado o modificado durante la ejecución de un programa, únicamente puede ser leído.

Figura 20: Constantes en C++

```
const char PRIMERMES[10] = "ENERO";  
const int DIAS_DE_SEMANA = 7;  
const float FUERZA_GRAVEDAD = 9.8f;  
const double PI = 3.14159265359;  
const char ARROBA = '@';
```

Fuente: elaboración propia.

Unidad 2. Estructuras de control

A medida que los algoritmos se vuelven más complejos, es necesario incorporar otro tipo de estructuras que nos permitan generar bifurcaciones en los pasos que se van a describir, por ejemplo, si un valor es igual a cero, no podemos dividir por él, pero en caso de que sea distinto de cero sí. Entonces, podemos contemplar esta situación, utilizando alguna estructura de control para determinar la acción que se va a realizar.

Estas estructuras nos permiten controlar el flujo de ejecución. En otras palabras, necesitamos ser capaces de controlar las acciones del algoritmo que estamos creando.

Supongamos que tenemos un algoritmo que permite cambiar el color de la luz de una vitrina mediante control remoto. El evento de entrada sería presionar el botón del control remoto, el proceso sería el cambio de color de la luz y el resultado sería el nuevo color mostrado en la vitrina.

Ahora bien, ¿cómo sabe el algoritmo que tiene que cambiar el color cuando se presiona el botón y no en otro lado? ¿Cómo sabe que tiene que cambiar el color de la vitrina y no de otra cosa? Todo este tipo de decisiones tienen que estar plasmadas en el algoritmo y no dejar nada librado al azar. Esto se logra mediante las estructuras de control que veremos a continuación.

Los ejemplos en este módulo, serán diagramados en PSeInt y codificados en C++, cabe destacar que las estructuras de control son comunes en todos los lenguajes de programación.

A continuación, revisaremos las diferentes estructuras de control que podemos utilizar en el diseño de un algoritmo.

Mira el siguiente video que te ayudará a introducirte en el tema:

Video 1. Lógica de programación

Fuente: elaboración propia.

Tema 1. Condicionales

Los condicionales son instrucciones que evalúan la veracidad de una sentencia, por lo tanto, utilizan valores booleanos (verdadero o falso) para decidir si ejecuta o no una determinada sección de código.

La sentencia si / if

Si o if establece la evaluación de una variable o expresión cuyo valor o resultado es de tipo booleano, es decir, verdadero o falso. En el caso de que sea verdadero, ejecuta una sección de código. Si es falso, no la ejecuta y pasa a la siguiente sección.

Cabe destacar que en las condiciones que se van a evaluar en la instrucción si o if, es donde utilizaremos la lógica proposicional.

Figura 21: Estructura de control Si PSeInt

```
Algoritmo Condicional
    // Definir variables
    Definir num1, num2 Como Entero;

    // Lectura de variables
    Leer num1;
    Leer num2;

    //Estructura de Control SI
    Si num1 = num2 Entonces
        // Verdadero
        Escribir "Números Iguales"
    SiNo
        // Falso
        Escribir "Números Distintos"
    Fin Si

FinAlgoritmo
```

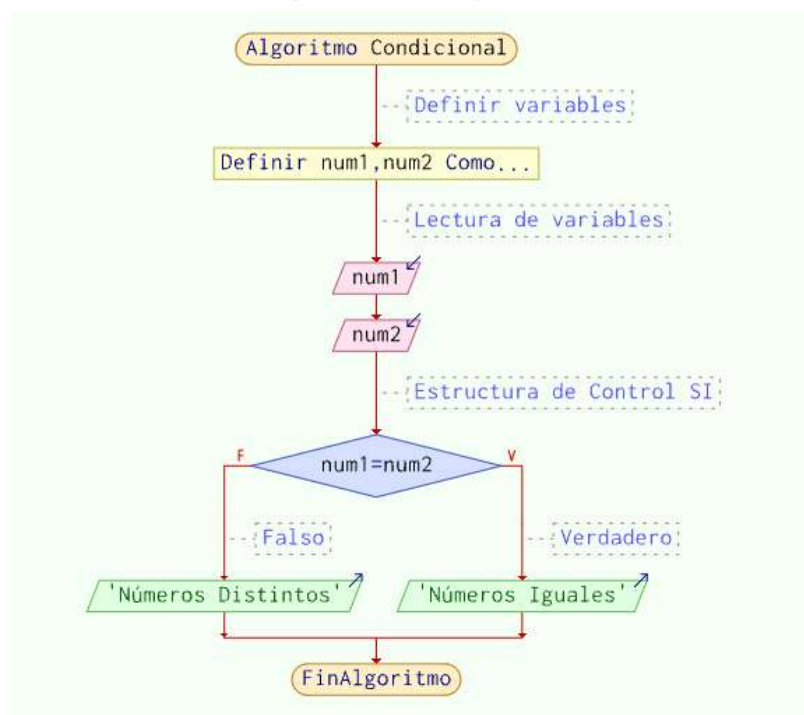

Fuente: elaboración propia con base en el software PSeInt (Novara, 2021).

Figura 22: Estructura de control If en C++

```
1
2  #include <iostream>
3
4  using namespace std;
5
6  int main()
7  {
8      int num1;
9      int num2;
10     cout << "Ingrese num1: " << endl;
11     cin >> num1;
12     cout << "Ingrese num2: " << endl;
13     cin >> num2;
14
15     if(num1 == num2)
16         cout << "Numeros iguales";
17     else
18         cout << "Numeros distintos";
19
20 }
```

Fuente: elaboración propia con base en el software C++.

Figura 23: Estructura de control Si. Diagrama de flujo



Fuente: elaboración propia con base en el software PSeInt (Novara, 2021).

Esta sentencia corresponde a una estructura de control que permite condicionar las instrucciones contenidas al resultado de la expresión evaluada. En el ejemplo, se evalúa que ambas variables leídas, num1 y num2, sean iguales. En caso de que esta validación sea verdadera, entonces, se escribe el mensaje.

Para comparar el valor de num1 y num2 se utilizan operadores lógicos de comparación, a continuación, se detallan algunos de ellos.

Tabla 2: Operadores lógicos de comparación

>	Mayor
<	Menor
<=	Menor o igual
==	Igual
!=	Distinto

Fuente: elaboración propia.

Las instrucciones contenidas dentro de la estructura Si / If en PSeInt se delimitan con la palabra FinSi.

La sentencia sino / else

Existe la posibilidad de indicar también una sección de código, en el caso de que la condición validada sea falsa, esto se incorpora después de la sección de código para la opción verdadera.

Con esta estructura, tendremos un camino para desarrollar en caso de que la condición sea verdadera o en caso de que sea falsa.

Para nuestro ejemplo, la instrucción que se va a ejecutar es el mensaje “Números Diferentes”, en caso de que la comparación num1 = num2 sea falsa.

Es importante destacar que la sentencia sino / else no existe por sí sola, siempre será un anexo para incluir a la instrucción si / if. Además, solo una de las secciones de código se ejecutará.

La sentencia sino-si/ else-if

Hasta ahora, las sentencias si / If y sino / else pueden ser de gran utilidad, pero ¿qué sucede si necesitamos evaluar más de dos condiciones? Por ejemplo, debemos realizar un algoritmo que identifique qué número es mayor que otro, además de indicar si son iguales.

Esto es posible mediante la instrucción sino-si / else-if.

A continuación, modificaremos el código anterior incluyendo esta nueva validación $\text{num1} > \text{num2}$. Se enviará el mensaje que el número 1 es mayor; si no, el número 2 es mayor.

Figura 24: Estructura de control Sino-Si PSeInt

```
Algoritmo Condicional
    // Definir variables
    Definir num1, num2 Como Entero;

    // Lectura de variables
    Leer num1;
    Leer num2;

    //Estructura de Control SI
    Si num1 = num2 Entonces
        // Verdadero
        Escribir "Números Iguales"
    SiNo
        Si num1 > num2 Entonces
            Escribir "Número 1 mayor a número 2"
        SiNo
            Escribir "Número 2 mayor a número 1"
        FinSi
    Fin Si
FinAlgoritmo
```

Fuente: elaboración propia.

Figura 25: Estructura de control Else-if C++

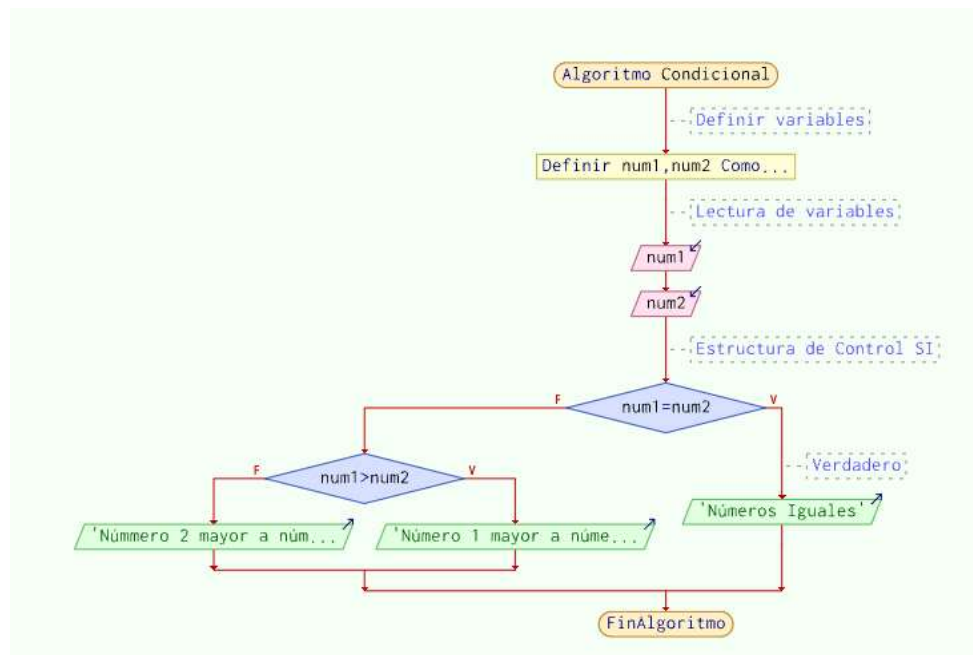
```

5
6 int main()
7 {
8     int num1;
9     int num2;
10    cout << "Ingrese num1: " << endl;
11    cin >> num1;
12    cout << "Ingrese num2: " << endl;
13    cin >> num2;
14
15    if(num1 == num2)
16        cout << "Numeros iguales";
17    else if(num1 > num2)
18        cout << "Numero 1 mayor a numero 2";
19    else
20        cout << "Numero 2 mayor a numero 1";
21
22    return 0;
23 }

```

Fuente: elaboración propia con base en el software C++.

Figura 26: Estructura de control Sino-Si. Diagrama de flujo



Fuente: elaboración propia con base en el software PSeInt (Novara, 2021).

En ambos casos, tenemos 2 comparaciones. La primera nos permite identificar los casos en los que los números son iguales; y la segunda, cuando comparamos si la variable num1 es mayor a num2. Si no, por defecto, indicamos que num2 es mayor que num1.

Como indicamos en un comienzo, las sintaxis son diferentes, pero las estructuras reflejan las mismas decisiones.

Solo uno de los caminos se ejecutará y, al igual que la sentencia sino / else, esta nueva instrucción sino-si / else-if existe solo si tenemos una sentencia si / if y siempre se incluye antes de la instrucción sino / else.

La sentencia Según / switch

Con según / switch podemos evaluar el valor de una variable y, de acuerdo con este, ejecutar cierta sección del código. Es similar a else if, ya que es útil cuando queremos comparar el valor de una variable con más de una opción. Switch y else-if cumplen la misma función.

Siempre es recomendado utilizar switch, en vez de else-if, debido a su simplicidad.

Por ejemplo, si queremos mostrar el autor de un libro específico, podemos utilizar la sentencia según / switch.

Figura 27: Estructura de control Según PSeInt

```
Algoritmo saludos
//Definir variables
Definir opcion Como Entero;

Escribir "Selecciona una opción:";
Escribir "1. Inglés";
Escribir "2. Español";
Escribir "3. Francés";

//Leer variable
Leer opcion;

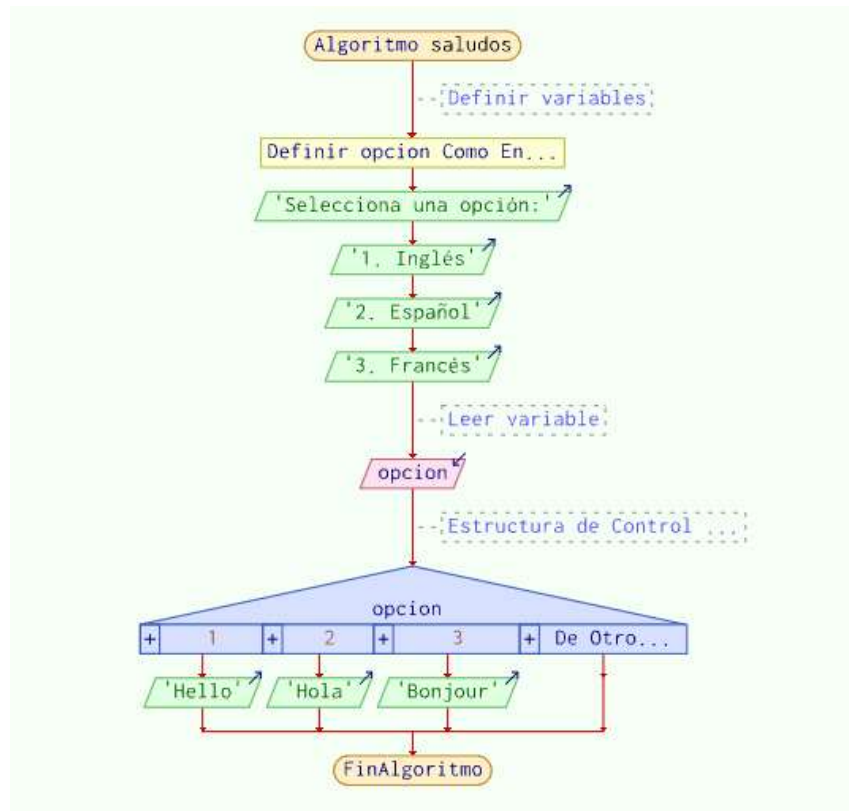
//Estructura de Control Según
Segun opcion Hacer
1:
    Escribir "Hello";
2:
    Escribir "Hola";
3:
    Escribir "Bonjour";

Fin Segun

FinAlgoritmo
```

Fuente: elaboración propia con base en el software PSeInt (Novara, 2021).

Figura 28: Estructura de control según. Diagrama de flujo



Fuente: elaboración propia con base en el software PSeInt (Novara, 2021).

Figura 29: Estructura de control Switch C++

```
6  int main()
7  {
8      int opcion;
9
10     cout <<"Ingrese opcion" <<endl;
11     cout<<"1. Ingles"<<endl;
12     cout<<"2. Español"<<endl;
13     cout<<"1. Frances"<<endl;
14
15     cin>>opcion;
16
17     switch(opcion){
18         case 1:
19             cout<<"Hello";
20             break;
21
22         case 2:
23             cout<<"Hola";
24             break;
25
26         case 3:
27             cout<<"Bonjour";
28             break;
29     }
30 }
```

Fuente: elaboración propia con base en el software PSeInt (Novara, 2021).

Según /switch permite agregar una opción de otro modo / default, es decir, una opción que se ejecute cuando ninguna de las demás es verdadera (también puede ser nombrada opción por defecto o default). Esto permite responder, en caso de que el valor de la variable no corresponda con ninguna de las cuatro provincias. Por ejemplo, podríamos imprimir un mensaje que muestre “No es una opción correcta”.

Figura 30: Estructura de control de otro modo PSeInt

```
Algoritmo saludos
    //Definir variables
    Definir opcion Como Entero;

    Escribir "Selecciona una opción:";
    Escribir "1. Inglés";
    Escribir "2. Español";
    Escribir "3. Francés";

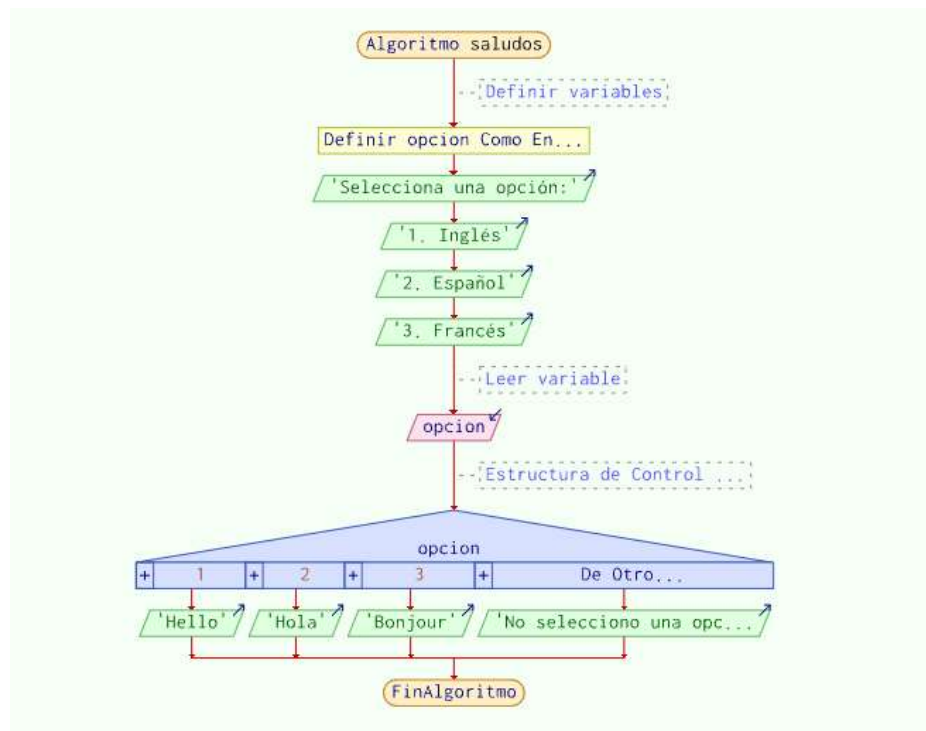
    //Leer variable
    Leer opcion;

    //Estructura de Control Según
    Segun opcion Hacer
    1:
        Escribir "Hello";
    2:
        Escribir "Hola";
    3:
        Escribir "Bonjour";
    De Otro Modo:
        Escribir "No selecciono una opción valida";
    Fin Segun

FinAlgoritmo
```

Fuente: elaboración propia con base en el software PSeInt (Novara, 2021).

Figura 31: Estructura de control de otro modo. Diagrama de flujo



Fuente: elaboración propia con base en el software PSeInt (Novara, 2021).

La cláusula de otro modo / default siempre va como opción final del según / switch

Figura 32: Estructura de control De otro modo

```

6  int main()
7  {
8      int opcion;
9
10     cout << "Ingrese opcion" << endl;
11     cout << "1. Ingles" << endl;
12     cout << "2. Español" << endl;
13     cout << "1. Frances" << endl;
14
15     cin >> opcion;
16
17     switch(opcion){
18         case 1:
19             cout << "Hello";
20             break;
21
22         case 2:
23             cout << "Hola";
24             break;
25
26         case 3:
27             cout << "Bonjour";
28             break;
29
30         default:
31             cout << "No es una opcion valida";
32             break;
33     }
34 }

```

Fuente: elaboración propia.

Tema 2. Ciclos o bucles

Un ciclo o bucle es una sección de código en la cual la ejecución se repite de acuerdo con una condición. Es similar al si o if, pero con la capacidad de ejecutar varias veces la misma sección de código. Son muy útiles cuando queremos recorrer una lista de elementos o ejecutar una sección de código indefinidamente.

El bucle para / for

Este bucle permite la repetición de una sección de código de acuerdo con el valor de una variable o expresión booleana. En el caso de que sea verdadera, el código se repite. En caso de que sea falsa, el código no se repite y pasa a la línea siguiente.

Se utiliza, comúnmente, para controlar la cantidad de veces que se necesita repetir el fragmento de

instrucciones contenidas en el ciclo.

Por ejemplo, queremos mostrar los números del 1 al 10.

Figura 33: Estructura repetitiva Para PSeInt

```
Algoritmo ciclo_para
    //Definir variables
    Definir i Como Entero;

    //Definir ciclo Para
    Para i <- 0 Hasta 10 Con Paso 1 Hacer
        ..... Escribir "Iteracion:" , i;
    Fin Para

FinAlgoritmo
```

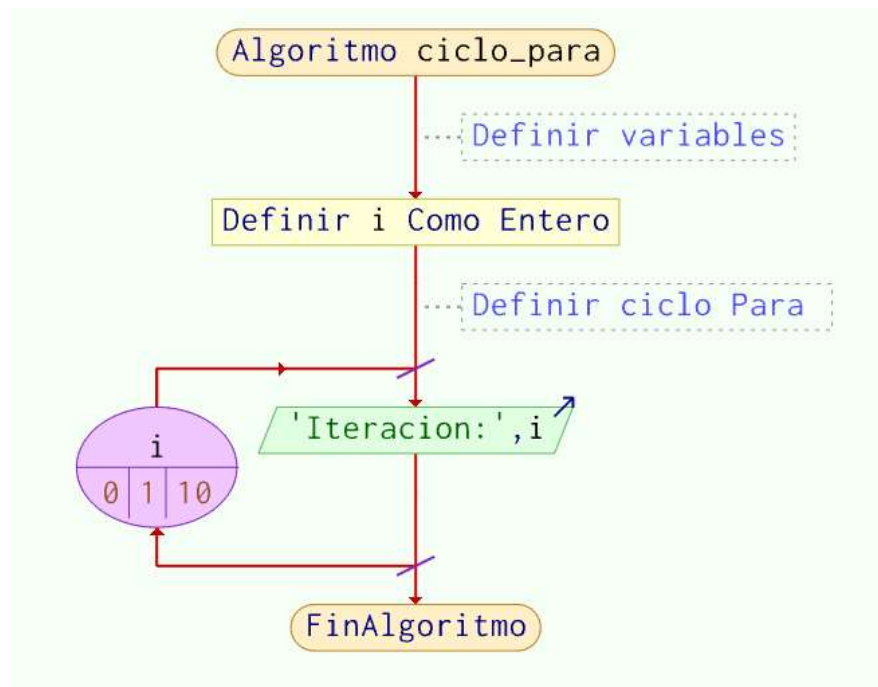
Fuente: elaboración propia con base en el software PSeInt (Novara, 2021).

Figura 34: Resultado estructura repetitiva Para PSeInt

```
*** Ejecución Iniciada. ***
i
Iteracion:0
Iteracion:1
Iteracion:2
Iteracion:3
Iteracion:4
Iteracion:5
Iteracion:6
Iteracion:7
Iteracion:8
Iteracion:9
Iteracion:10
*** Ejecución Finalizada. ***
```

Fuente: elaboración propia con base en el software PSeInt (Novara, 2021).

Figura 35: Estructura repetitiva para. Diagrama de flujo



Fuente: elaboración propia con base en el software PSeInt (Novara, 2021).

En el caso de PSeInt, la variable *i* parte en el número 0, porque le especificamos que así sea, también definimos de cuanto en cuanto se incrementa.

Se ejecutan secuencialmente la cantidad de iteraciones indicadas en la sentencia, en este caso, corresponde a 10 iteraciones.

Figura 36: Estructura repetitiva para en c++

```

int main()
{
    for (int i = 0; i <= 10; i++) {
        cout << i << "\n";
    }

    return 0;
}

```

Fuente: elaboración propia.

La sentencia «cout» realiza la misma tarea que la sentencia «printf», ambas imprimen a la salida estándar, que en este caso sería la pantalla.

El bucle Mientras / while

Mientras / while es otro tipo de bucle que, al igual que para / for, permite la repetición de ejecución en un bloque de código determinado. Una de las diferencias de mientras / while es que no se puede definir una variable ni cambiar su valor en la misma línea, como en el caso de para / for. Si no que se ejecuta mientras la condición sea verdadera.

Este bucle o ciclo se utiliza cuando existe un nivel de incertidumbre en la condición, pues no siempre sabemos la cantidad de iteraciones que se ejecutarán.

Por ejemplo, realizaremos un algoritmo que nos permita sumar 1 unidad a un número que comienza en 1, mientras el resultado no supere el tope solicitado por el usuario.

Figura 37: Estructura repetitiva Mientras PSeInt

```
Algoritmo ciclo_mientras
    //Definir variables
    Definir num, tipo Como Entero;

    //Asignar variable
    num <- 1;

    //Solicitar el tope
    Escribir "Ingrese el tope a considerar:"
    Leer tope;

    //Estructura repetitiva Mientras
    Mientras num ≤ tope Hacer
    .....
        Escribir num;
        num <- num + 1;
    .....
    Fin Mientras

FinAlgoritmo
```

Fuente: elaboración propia con base en el software PSeInt (Novara, 2021).

Figura 38: Resultado estructura repetitiva Mientras PSeInt

```
*** Ejecución Iniciada. ***  
Ingrese el tope a considerar:  
> 7  
1  
2  
3  
4  
5  
6  
7  
*** Ejecución Finalizada. ***
```

Fuente: elaboración propia con base en el software PSeInt (Novara, 2021).

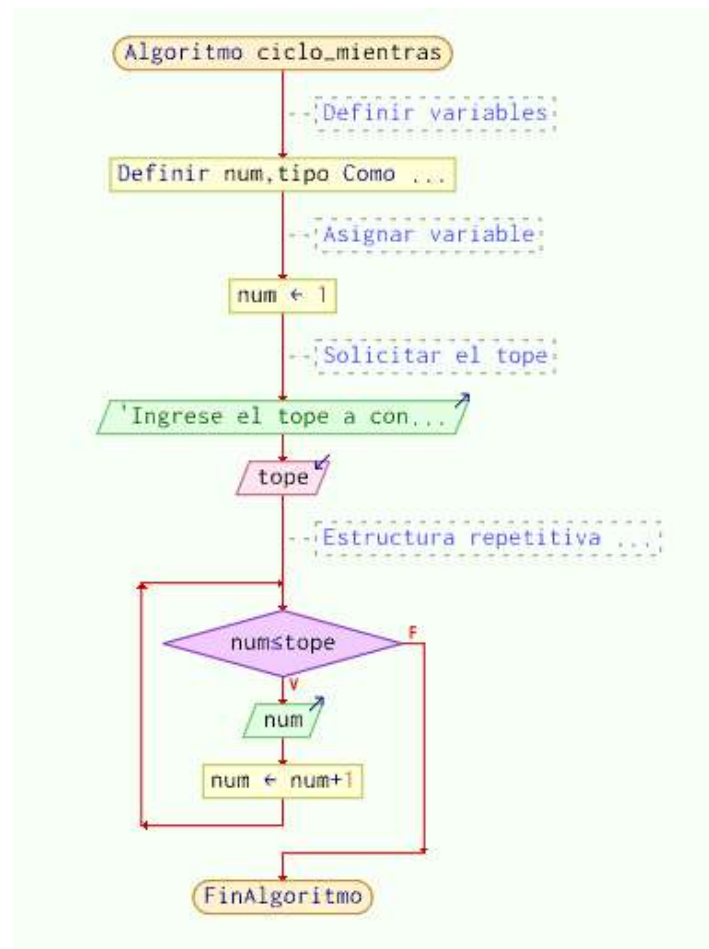
Figura 39: Estructura repetitiva Mientras C++

```
#include <stdio.h>  
#include <iostream>  
using namespace std;  
  
int main()  
{  
    int num, tope, i;  
    cout << "Ingrese el tope:";  
    cin >> tope;  
  
    i = 1;  
    while (i <= tope) {  
        cout << i << "\n";  
        i++;  
    }  
  
    return 0;  
}
```

Fuente: elaboración propia con base en el software C++.

Si bien las estructuras repetitivas en definitiva son estructuras de control, las mencionamos así para evitar equivocaciones.

Figura 40: Estructura repetitivas Mientras. Diagrama de flujo



Fuente: elaboración propia con base en el software PSeInt (Novara, 2021).

El bucle repetir / do-while

Este bucle cumple las mismas funciones que los ciclos anteriores (permite la repetición de la ejecución en una sección de código), pero con la diferencia de que asegura que el código se repita, al menos, una vez. Lo que hace es ejecutar el código que se va a repetir y luego evaluar si lo tiene que volver a ejecutar. Los ciclos anteriores siempre evalúan al iniciar el bucle.

En el ejemplo, jugaremos al número secreto, es decir, tendremos un número secreto que el usuario debe adivinar, le preguntaremos al usuario un número hasta que este sea igual al número secreto.

Figura 41: Estructura repetitiva Repetir PSeInt


```

Algoritmo ciclo_repetir
    //Definir variables
    Definir numSecreto, numUsuario Como Entero;

    //Definir valor de numero secreto
    numSecreto ← 7;

    //Estructura Repetir
    Repetir
        Escribir "Adivina número secreto";
        Leer numUsuario;

    Hasta Que numSecreto = numUsuario;

    Escribir "Adivinaste !!!";

FinAlgoritmo

```

Fuente: elaboración propia con base en el software PSeInt (Novara, 2021).

Figura 42: Resultado estructura repetitiva Repetir PSeInt

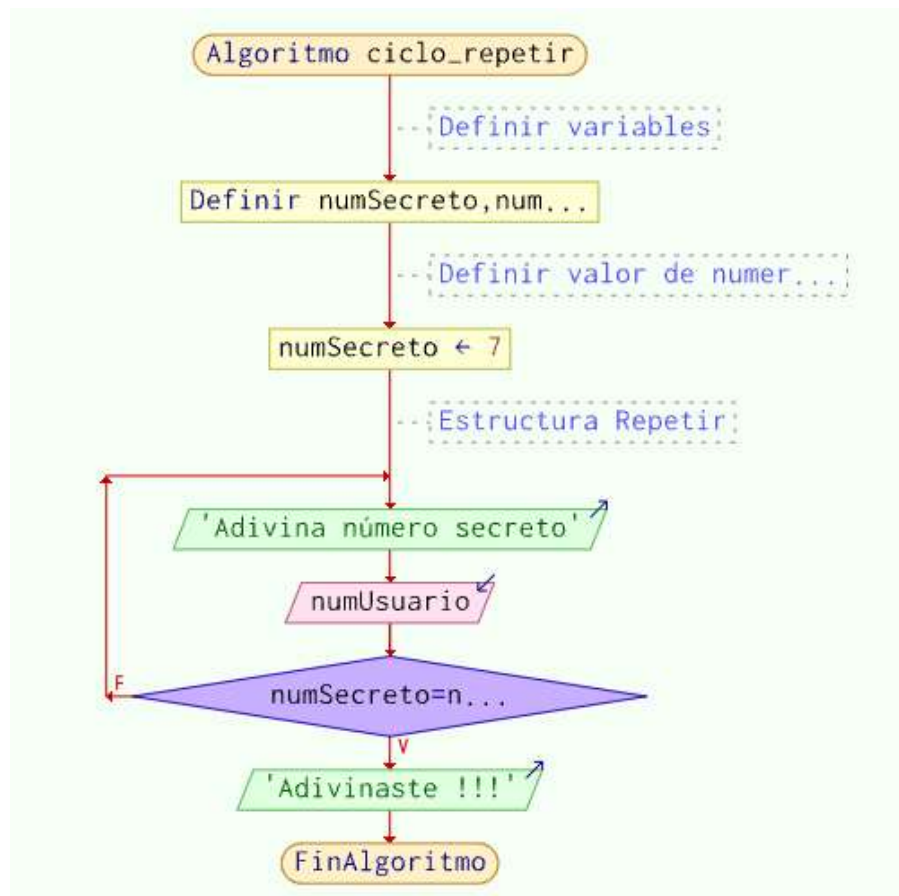
```

*** Ejecución Iniciada. ***
Adivina número secreto
> 1
Adivina número secreto
> 4
Adivina número secreto
> 7
Adivinaste !!!
*** Ejecución Finalizada. ***

```

Fuente: elaboración propia con base en el software PSeInt (Novara, 2021).

Figura 43: Estructura de control Repetir. Diagrama de flujo



Fuente: elaboración propia con base en el software PSeInt (Novara, 2021).

Figura 44: Estructura de control Repetir en C++

```

int main()
{
    int numSecreto, numUsuario;

    numSecreto = 7;
    do {
        cout << "Adivina numero secreto \n";
        cin >> numUsuario;
    } while (numUsuario != numSecreto);

    cout << "Adivinaste !!!";

    return 0;
}
  
```

Fuente: elaboración propia.

El bucle for each

Este ciclo es específico para recorrer estructuras de manejo de datos, que revisaremos en el siguiente módulo.

Tema 3. Funciones y funciones anónimas

Hay muchas formas de codificar un programa y en el desarrollo de software no existe una única forma de construir el componente de código. Ahora bien, sí hay lineamientos y guías para que ese desarrollo pueda ser reutilizado (ahorrando costos, ya que no tenemos que gastar tiempo haciendo las cosas de nuevo) y extensible (la capacidad de agregarle funcionalidad de manera fácil).

Uno de esos lineamientos es crear funciones. Las funciones son secciones de código que podemos reutilizar, de forma tal que no tengamos que codificar varias veces el mismo funcionamiento y lo podamos reutilizar.

Por ejemplo, podemos tener una función que nos permita mostrar un menú por pantalla.

Figura 45: Función Menú PSeInt

```
Funcion opcion <- func_menu
    Escribir "Menu";
    Escribir " ";
    Escribir "1. Sumar";
    Escribir "2. Restar";
    Escribir "3. Multiplicar";
    Escribir "4. Dividir";
    Escribir "5. Salir";

    Leer opcion;
Fin Funcion
```

Fuente: elaboración propia con base en el software PSeInt (Novara, 2021).

Para utilizar este fragmento de código o de algoritmo, debemos hacer una llamada a dicha función. Esto se realiza solo indicando el nombre de la función seguido de paréntesis.

Figura 46: Llamada función PSeInt

```
Funcion opcion <- func_menu
  Escribir "Menu";
  Escribir "";
  Escribir "1. Sumar";
  Escribir "2. Restar";
  Escribir "3. Multiplicar";
  Escribir "4. Dividir";
  Escribir "5. Salir";

  Leer opcion;
Fin Funcion

Algoritmo calculadora
  seleccion = func_menu;
  Escribir seleccion;
FinAlgoritmo
```

Fuente: elaboración propia con base en el software PSeInt (Novara, 2021).

El resultado de esta llamada queda de la siguiente manera.

Figura 47: Resultado función Menú PSeInt

```
*** Ejecución Iniciada. ***
Menu

1. Sumar
2. Restar
3. Multiplicar
4. Dividir
5. Salir
> 1
1
*** Ejecución Finalizada. ***
```

Fuente: elaboración propia con base en el software PSeInt (Novara, 2021).

Una de las ventajas de crear funciones es que podemos llamar a este fragmento de código desde diversas partes de nuestro algoritmo, pero si se quiere hacer un cambio o incluir un código, esto se realiza solo una vez dentro de la función.

De esta manera, cuando necesitemos agregar o cambiar el formato de cómo se muestran los mensajes, solo tenemos que cambiar la función Imprimir y no todas las líneas de código donde mostramos el menú.

Tema 4. Recursividad

Llamar una función

En programación, llamar o invocar a una función significa utilizarla, es decir, agregar el código de dicha función al flujo de ejecución del programa.

La recursividad es una característica de los lenguajes de programación que permite que un subprograma se invoque a sí mismo. La recursividad es útil para resolver problemas definibles en sus propios términos; y es, en cierta medida, análoga al principio de inducción.

La recursividad se debe usar cuando sea absolutamente necesaria, es decir, cuando no exista una solución iterativa simple. Utilizando mal la característica de recursividad podemos caer en bucles infinitos o utilización de los recursos computacionales de manera inadecuada.

Son comunes para expresar series matemáticas cuando se necesitan en una aplicación específica.

Estas funciones tienen 2 características importantes: la función debe llamarse a sí misma dentro del código.

Debe existir un caso base que nos permita detener la ejecución.

Figura 48: Recursividad PSeInt

```

Funcion val <- factorial ( num )
    //Definicion de variables
    Definir val Como Entero;

    Si num = 1 Entonces
        //Caso base
        val <- 1;
    SiNo
        //Llamada recursiva
        val <- factorial(num - 1)
    FinSi

Fin Funcion

```

Algoritmo recursividad

```

    resultado = factorial(3);
    Escribir resultado;

```

FinAlgoritmo

Fuente: elaboración propia con base en el software PSeInt (Novara, 2021).

A continuación, podrás ver cómo impactan los conceptos en Java:



Fuente: elaboración propia.

Generá el diagrama de flujo y pseudocódigo para cada una de las siguientes solicitudes.

1. Solicitar el ingreso de un número y devolver si el número es par, impar o cero
2. Ciclo iterativo que divide un número por 2, se ejecutará mientras el resultado sea mayor a 0.
3. Ciclo iterativo que multiplica un número por 3, 11 veces.
4. Ciclo iterativo que imprime un menú con las siguientes opciones, 1. entrada, 2. plato principal, 3. postre y 4. salir. Se ejecutará mientras no seleccionen la opción salir.
5. Ciclo condicional que permita identificar mayores y menores de edad. Es decir. Mayores cuando superan los 18 años de edad, menores en el resto de los casos.

6. Ciclo condicional que entregue la capital de los siguientes países: (1) España, (2) Francia, (3) Canadá.

Podrás descargar las resoluciones en el siguiente PDF:



Fuente: elaboración propia.

Video de habilidades

¿Qué indica las secciones entre estos símbolos “`/** */`” en Java?

Comentarios

Objetos

Métodos

Clases

Atributos

Justificación

¿Qué palabra clave identifica una sección de código como una clase en Java?

Object

New

String

Class

Interface

Justificación

¿Para qué sirve el constructor de una clase en Java?

Definir una clase

Imprimir en pantalla

Inicializar los objetos que se crean

No existen los constructores

Para crear interfaces

Justificación

¿Cómo identificamos un constructor de la clase en Java?

Tiene la palabra class antes de su nombre

Es un método estático

Definiendo una interfaz

Es un método que tiene el mismo nombre que la clase

Declarando atributos

Justificación

¿Cómo suele definirse el nombre de una clase?

Con espacios

Inicia con una mayúscula y luego separando cada palabra con mayúsculas, sin guiones ni espacios

Con guiones

Todo en minúscula

Puede definirse como uno quiera, no hay recomendaciones ni restricciones

Justificación

Cuadernillo de actividades

Actividad

Resolución

Glosario

Referencias

Joyanes Aguilar, L. (2007). Fundamentos de programación. McGraw-Hill.

Novara, P. (2021). PSeInt [Software intérprete de pseudocódigo]. AR.

